

UNIVERSIDAD NACIONAL DE COLOMBIA
Facultad de Ingeniería
Departamento de Ingeniería de Sistemas e Industrial

Programación Dinámica

Matemáticas Discretas I

Diego Alejandro Robayo Córdoba

Docente: Jhoan Sebastián Tenjo García

Bogotá, Colombia
19 de julio de 2026

1. Introducción

En este documentose describe el funcionamiento y las características algoritmicas de diez problemas de programación dinámica, donde se incluye Criptografía, grafos, álgebra de Boole, Shannon.

2. Cifrado César: romper un mensaje antiguo

El cifrado de César consiste en modelar el alfabeto como un conjunto de índices de 0 a 25, donde cada letra se reemplaza por otra que se encuentra un número fijo de posiciones más adelante en el alfabeto. Por ejemplo, si el desplazamiento es 3, la letra A se reemplaza por D, B por E, y así sucesivamente. Esta expresión se expresa mediante aritmética modular de la forma

$$(x + k) \text{ mód } 26$$

donde x es el índice de la letra original, k es el desplazamiento y 26 es el número de letras en el alfabeto. De esta forma, si el índice se pasa de 26, con el modulo se vuelve al inicio del alfabeto para recorrerlo nuevamente.

En cuanto el descifrado, se hace la operación inversa, es decir, se resta el desplazamiento. Por ejemplo, si el desplazamiento es 3, luego del descifrado la letra D se reemplaza por A, E por B, y así sucesivamente. Esta expresión se expresa mediante aritmética modular de la forma

$$((x + k) - k) \text{ mód } 26 = x$$

Debido al espacio de claves tan pequeño que tiene el alfabeto, el cifrado de César es muy fácil de romper en un ataque de fuerza bruta. Por esto es posible probar todas las claves posibles hasta encontrar un texto legible. Si tomamos como ejemplo el mensaje cifrado "KROD XQDO", podemos probar todas las claves posibles y obtener el siguiente resultado:

$$k = 0 : KROD \quad XQDO$$

$$k = 1 : JQNC \quad WPCN$$

$$k = 2 : IPMB \quad VOBM$$

$$k = 3 : HOLA \quad UNAL$$

3. RSA de juguete: llaves, cifrado y descifrado

Primero se seleccionan dos números primos p y q . A partir de ellos se calcula $n = p \times q$. Luego se calcula la función ϕ de Euler del numero n . Como n es el producto de dos primos, entonces

$$\phi(n) = (p - 1)(q - 1)$$

Esta función indica el número de enteros positivos menores que n que son coprimos con n . Y es fundamental para la clave privada.

Se proporciona el exponente e , que solo es valido si se cumple $\gcd(e, \phi(n)) = 1$. Es decir, e debe ser coprimo con $\phi(n)$. En la implementación esta verificación se realiza mediante el algoritmo de Euclides.

En el cálculo del inverso modular, se utiliza el algoritmo extendido de Euclides para encontrar el valor del exponente privado d tal que

$$ed \equiv 1 \text{ mód } \phi(n)$$

este número constituye la clave privada.

El cifrado de un mensaje M se obtiene mediante $C \equiv M^e \pmod{n}$. La potenciación modular garantiza que el resultado se mantenga entre 0 y $n - 1$. El descifrado se realiza mediante $M \equiv C^d \pmod{n}$. De esta forma el resultado coincide con el mensaje original.

Podemos asegurar que RSA funciona porque el exponente privado d es el inverso modular del exponente público e . Esto significa que

$$ed \equiv 1 \pmod{\phi(n)}$$

De acuerdo con el teorema de Euler, si M es coprimo con n , entonces

$$M^{\phi(n)} \equiv 1 \pmod{n}$$

Como consecuencia,

$$(C)^d = (M^e)^d = M^{ed} \equiv M \pmod{n}$$

lo que garantiza que el descifrado del mensaje cifrado C nos devuelve el mensaje exactamente el mensaje original M .

4. MPC básico: calcular un promedio sin mostrar los datos

Cada nota se divide en tres partes aleatorias. Para una nota x , se generan valores aleatorios s_1 y s_2 . La tercera parte se calcula como $s_3 \equiv x - s_1 - s_2 \pmod{M}$, donde M es un número muy grande. De esta forma se garantiza que

$$x \equiv s_1 + s_2 + s_3 \pmod{M}$$

Cada servidor recibe una de las partes aleatorias. Por ejemplo, si una nota es 40, podría dividirse como

Nota	Servidor 1	Servidor 2	Servidor 3
40	132456	654321	213269

El servidor 1 conoce únicamente 132456. Ese valor fue generado aleatoriamente y no tiene relación con la nota original, por lo que no es posible deducir la nota original. Lo mismo ocurre con los otros servidores.

Para la reconstrucción de la suma, cada servidor envía la suma de todas sus partes. Entonces la suma total de las notas se obtiene mediante

$$Suma \equiv S_1 + S_2 + S_3 \pmod{M}$$

Finalmente, el promedio se calcula de la forma habitual, dividiendo la suma total entre el número de notas. De esta forma, es posible calcular el promedio sin revelar las notas individuales.

5. MPC básico: calcular un promedio sin mostrar los datos

Una red de transporte puede modelarse mediante un grafo ponderado $G = (V, E)$, donde V es el conjunto de nodos y E es el conjunto de aristas. Cada arista $(u, v) \in E$ tiene un peso asociado $w(u, v)$ que representa el costo o la distancia entre los nodos u y v . El objetivo es encontrar el camino más corto desde un nodo de origen s hasta un nodo de destino t . Si un camino está formado

por las aristas $(u_1, u_2), (u_2, u_3), \dots, (u_{k-1}, u_k)$, entonces el costo total del camino es la suma de los pesos de las aristas que lo componen:

$$d(u_1, u_k) = \sum_{i=0}^{k-1} w(u_i, u_{i+1})$$

El algoritmo busca minimizar esta suma para encontrar el camino más corto entre los nodos de origen y destino.

Dijkstra mantiene para cada vértice una estimación de la distancia más corta desde el nodo de origen. Inicialmente, la distancia al nodo de origen es cero y la distancia a todos los demás nodos es infinita. A medida que se exploran los nodos, se actualizan las distancias si se encuentra un camino más corto. Para esto el algoritmo realiza dos pasos:

1. Seleccionar el nodo con la distancia más corta conocida y marcarlo como visitado.
2. Actualizar las distancias de los nodos adyacentes al nodo seleccionado, si se encuentra un camino más corto a través de él.

Si desde un vertice u existe una arista hacia un vertice v , se verifica si el camino

$$d(u) + w(u, v)$$

es mejor que la distancia actualmente conocida para v . Si es así, se actualiza la distancia de v y se registra que el camino más corto hacia v pasa por u :

$$d(v) = d(u) + w(u, v)$$

Al mismo tiempo se guarda que el predecesor de v es u , lo que posteriormente permite reconstruir la ruta completa.

El algoritmo supone que, una vez seleccionado el vértice con la menor distancia conocida, esa distancia ya es la mínima posible y nunca volverá a mejorar. Esta propiedad únicamente se cumple cuando todas las aristas poseen pesos no negativos. Si existieran pesos negativos, podría aparecer posteriormente un camino más corto hacia un vértice que ya había sido considerado definitivo, invalidando el algoritmo. Por esta razón Dijkstra únicamente garantiza resultados correctos en grafos cuyos pesos sean mayores o iguales que cero.

Además, un camino es optimo cuando su costo total es menor o igual que el de cualquier otro camino entre el mismo origen y destino. Es decir, si P^* es un camino encontrado por dijkstra, entonces para cualquier otro camino P se cumple

$$Costo(P^*) \leq Costo(P)$$

Esto garantiza que la distancia obtenida corresponde a la ruta más corta disponible en el grafo.